

FOL: Bridging Object-Oriented and Functional Programming via the Metaobject Protocol

Frank Adrian
Ancar Technology
Olathe, KS, USA

frank.adrian314159@gmail.com

Abstract

Hickey [19] omitted CLOS from Clojure because its mutable objects are a complexity hazard incompatible with immutability. FOL (Functional Object Lisp) uses the Metaobject Protocol (MOP) to back instances with persistent structures, preserving CLOS’s open dispatch model while enforcing immutable reference semantics. FOL contributes a persistent metaclass with hybrid storage and lazy migration, and a specificity-ordered predicate dispatch model.

Slot reads incur 1.2–2.0× overhead; slot updates cost 30–248× in isolation. At idiomatic workload scales, FOL is 7–43× slower than mutable CLOS; naive direct translation of mutable algorithms reaches 96× (persistent Quicksort); however, at extreme scale persistent state maps avoid the hash-table fragmentation that degrades the mutable baseline, yielding a 3.8× runtime advantage in logic simulation. Predicate dispatch uses a level-based ordering sorted once at load time in $O(ND \log N)$ (D = hierarchy depth), avoiding the pairwise theorem-prover overhead of subsumption systems; the trade-off is an $O(N)$ per-call linear scan (vs. $O(1)$ cached dispatch), with closed defn dispatch matching hand-written cond cascade performance.

CCS Concepts

• **Software and its engineering** → **Constraints; Classes and objects; Functional languages; Object oriented languages; Extensible languages.**

Keywords

Functional programming, object-oriented programming, Lisp, persistent data structures, CLOS, MOP

1 Introduction

Common Lisp’s object system (CLOS) is extensible, but its mutable objects create three problems.

P1 — Speculative update cost. Speculative workloads must copy objects or maintain undo logs. Persistent alternatives (FSet [5], Sycamore [10]) lack CLOS integration and generic-function dispatch. Isolated slot updates cost 30–248×, but $O(\log n)$ structural sharing bounds per-update cost regardless of the number of live objects in the system; at sufficient scale, persistent collections used as instance state maps avoid the hash-table fragmentation that degrades mutable baselines (Section 5.5).

P2 — Schema evolution labor. Renaming slots requires manual update-instance-for-redefined-class methods; the standard hook lacks automatic forwarding. FOL’s `:alias` annotation recovers renamed slots at first access, composing multi-hop chains in a single pass (Section 5.2.2).

P3 — Inexpressive predicate dispatch. CLOS dispatch is type-limited; value-level conditions require manual guards, losing extensibility. `filtered-functions` [26] extends this via subsumption but requires pairwise logical-implication checks over Boolean predicate expressions. FOL’s syntactic ordering is decidable at load time in $O(ND \log N)$; per-call overhead matches manual cond cascades (1.6×; Section 5.2.1).

FOL (Functional Object Lisp) addresses these problems via the MOP [20]: class instances are backed by persistent data structures, and specificity-ordered predicate dispatch integrates with standard method combinations. Slot mutation is replaced by functional assoc; other CLOS interfaces behave normally (Table 1). The contributions compose: Listing 4 gates on runtime values while reading persistent state, while Listing 3 enables $O(1)$ update discard without copying.¹

The contributions of this paper are:

- (1) A **persistent metaclass system** (Section 2) with a hybrid native/trie storage strategy and lazy alias-based schema migration via multi-hop chain replay (Section 4.2).
- (2) A **specificity-ordered predicate dispatch model** (Section 4): a level-based ordering relation that is decidable at load time without theorem-prover calls, with integration into standard CLOS method combinations.

2 Architecture

Slot storage uses a hybrid layout: objects with ≤ 8 slots use native slots; wider objects use a 32-way trie. Most classes (80%) have ≤ 8 slots [21]. Redefinitions are lazy: renamed slots resolve via alias maps; additions evaluate `:initforms`. If added slots push a native object > 8 slots, its next mutation converts it to a trie layout.

The persistent-class metaclass ensures post-initialization immutability; standard mutations raise errors. `assoc` returns a fresh instance sharing unchanged trie nodes via path copying. Snapshots decouple state from identity.

2.1 Collections and Runtime

FOL implements persistent **Vectors**, **Maps/Sets**, and **Sorted collections** using 32-way tries [3, 24]. Branching factor 32 balances tree depth against cache-line node size. FOL transpiles to Common Lisp via expansion and code generation phases.

¹FOL addresses both dimensions of the Expression Problem [31] within a dynamically typed setting: new generic functions can be added over existing classes, and new clauses can be added for new types, without modifying existing code. Static type safety guarantees are absent by design.

Table 1: MOP Protocol Adaptation

Protocol	Status	Rationale
slot-value-using-class	Redirected	Reads from base object or overflow trie
(setf slot-value-using-class)	Guarded	Blocks post-init writes
slot-boundp-using-class	Redirected	Checks object slot membership
slot-makunbound-using-class	Blocked	Immutability invariant — use dissoc to unbind slots
validate-superclass	Extended	Cross-metaclass mixing
initialize-instance	Extended	Populates base object and overflow trie from initargs; stamps %schema-version (an integer matching the class’s current version counter) at birth, enabling update-instance-for-redefined-class to detect stale instances
reinitialize-instance	Extended (:before on metaclass)	Snapshots current alias-map, overflow indices, and version counter into a class-version struct before each redefinition; advances the version counter
compute-slots	Specialized	Implements hybrid native/trie layout
update-instance-for-redefined-class	Implemented (:after)	Multi-hop chain replay: composes alias-maps from version chain back to birth version, recovering renamed values (Sec. 4.2)
change-class	Omitted	Changing a live object’s class violates immutability; callers construct a new instance of the target class instead (requires compatible metaclasses)

Method combination uses standard CLOS execution semantics (:before/:after/:around). $\prec_{disp}$ replaces the class precedence list; as a total order, it preserves CLOS’s most/least-specific-first guarantees even if ties are broken by definition index.

2.2 Space Complexity

Collections and identity types (Atoms, Refs, Agents) hold only current values; old snapshots are GC-eligible unless referenced. Each assoc copies one root-to-leaf path ($O(\log_{32} N)$ nodes), so allocation per update is bounded by trie depth rather than collection size. SBCL nursery reclamation keeps GC overhead at $\approx 1.5\%$ in the LSim benchmark (profiled at the 8x32x32-9000 configuration via sb-sprof).

3 Features

FOL adopts Clojure’s literal syntax and sequence APIs. **Transducers** decouple algorithm logic; **Lazy sequences** defer computation; **Atoms, Refs, and Agents** handle state transitions; and **Transient builders** reduce bulk update overhead. **Macros** follow Clojure-style expansion. Generic functions dispatch by arity, then specificity (Section 4). **Closed dispatch** (defn) emits a static cond cascade; **open dispatch** (defmethod) supports CLOS method combinations and extensibility. Programmers must choose one per function name. Table 2 summarises the decision criteria.

Table 2: Choosing defn vs. defmethod

Requirement	defn	defmethod
Full clause set known at definition time	✓	✓
Downstream code may add clauses	×	✓
call-next-method needed	×	✓
:before/:after/:around qualifiers	×	✓
Minimum dispatch overhead (static cond)	✓	×

4 Predicate Dispatch

Figure 1 formalizes FOL’s multi-pattern dispatch syntax.

4.1 Dispatch Ordering Rules

FOL’s predicate specializers (var (fn args...)) evaluate at dispatch time; predicates are more specific than types. Specificity categories (Table 3) resolve ties via definition order. $\prec_{disp}$ conservatively approximates semantic specificity: Level 3 > 2 > 1 > 0. Level 2 uses semantic subclassing (Spec-Type); multi-parameter patterns apply Spec-Prefix and Spec-Tail position by position. Level 1 sets fixed-vs-rest precedence (Spec-Fixed). Consider:

```
(defn classify
  ([x <integer>] :any-integer) ; catch all
  ([x (even?)] :even))        ; special case
```

FOL reorders this to even? first, so even integers never reach :any-integer. User-defined predicates must maintain totality. For closed defn groupings, syntactically decidable shadowed clauses trigger a compilation warning. Definition-order reliance creates modularity hazards; we are exploring explicit mechanisms like prefer-method.

Method qualifiers execute in CLOS-standard order (e.g., most-specific-first :before), but $\prec_{disp}$ replaces the CPL. This deliberately diverges: predicates execute before types, prioritizing behavioral constraints over structural inheritance.

Conjunction maps to its components’ maximum level, providing a conservative upper bound. Disjunction (or p q) merges distinct domains, making it semantically less specific than either

```

defn ::= (defn name (s-clause | m-clause)) | (fn (s-clause | m-clause))
defg/m ::= (defgeneric name...) | (defmethod name qualifier2 (s-clause | m-clause))
s-clause ::= [param*] body*
m-clause ::= {(symbol param | :keys [(symbol | (sym param))*]*} body*
param ::= symbol | (symbol type) | (symbol (fn arg*)) | destr
destr ::= [param*] | { :keys [(symbol | (sym param))*]* }
type ::= <type-name>
qualifier ::= :before | :after | :around

```

Figure 1: FOL Multi-Pattern Dispatch Grammar (compressed)

Table 3: Predicate Specificity Categories

Level	Category	Example
3	Predicates	(x (even?))
2	Types	(x <integer>)
1	Destructuring (all-wildcard)	[x y]
0	Wildcard	x

conjunct; the uniform level calculus cannot express this relationship. Negation is excluded as (not p)’s ordering requires knowing the complement domain.

Known deficiency — type-annotated conjunctions. (and (<integer> (prime?))) receives Level 3 ($\max(2, 3) = 3$), the same level as the bare predicate (prime?) alone, even though the conjunction is strictly more constrained. The syntactic calculus cannot determine this semantic subsumption without a theorem prover. The only current mitigation is definition order, which introduces a modularity hazard: if multiple libraries extend the same generic function with overlapping Level-3 predicates, the behavior depends on load order. A future composite level for type-annotated predicates (e.g., (x <integer> (prime?))) could recover automatic ordering without a theorem prover by treating the type as a static guard.

4.1.1 Formal Specificity Ordering. We formalize the specificity relation $\prec$ using the inference rules summarized in Table 4. Let $\mathcal{L}(p) \in \{0, 1, 2, 3\}$ be the specificity level of pattern p . **Syntactic form classes:** wildcard (wc), type annotation (ty, written <name>), predicate (pred, written (fn args...)), sequential (seq, written [...]), and map (map, written { ... }); sorted canonically via string< on symbol-names, keywords preceding symbols and strings). $p \sim q$ iff $\mathcal{L}(p) = \mathcal{L}(q)$ and p, q belong to the same form class. For compound sequential and map patterns, $\mathcal{L}([p_1, \dots, p_n]) = \max_i \mathcal{L}(p_i)$ and $\mathcal{L}(\{k_1 p_1, \dots\}) = \max_i \mathcal{L}(p_i)$: the level of a pattern list is the maximum element level, consistent with the conjunction rule. Spec-Level therefore applies to compound patterns as a whole: $[x \text{ (even?)}]$ (L=3) fires before $[x <integer>]$ (L=2) in its entirety. Position-by-position rules (Spec-Prefix, Spec-Tail) apply only when two compound patterns share the same max level (i.e., satisfy $p \sim q$); Spec-Level takes precedence otherwise. **Convention:** $p_1 \prec p_2$ means p_1 fires before p_2 —i.e., p_1 is more specific. Higher levels are more specific, so Spec-Level concludes $p_1 \prec p_2$ when $\mathcal{L}(p_1) > \mathcal{L}(p_2)$.

Rule application priority: Spec-Level takes precedence over all positional rules; within the same level, Spec-Type refines ordering

Table 4: Specificity Inference Rules

Rule	Formalization
Spec-Level	$\frac{\mathcal{L}(p_1) > \mathcal{L}(p_2)}{p_1 \prec p_2}$
Spec-Type	$\frac{C_1 \sqsubset C_2}{(x C_1) \prec (x C_2)}$
Spec-Prefix	$\frac{p_1 \prec q_1 \quad \mathcal{L}([p_1, \dots]) = \mathcal{L}([q_1, \dots])}{[p_1, \dots] \prec [q_1, \dots]}$
Spec-Tail	$\frac{p_1 \sim q_1 \quad [p_2, \dots] \prec [q_2, \dots]}{[p_1, \dots, p_n] \prec [q_1, \dots, q_n]}$
Spec-Fixed	$\frac{\mathcal{L}(p_i) = \mathcal{L}(q_i) \ \forall i \in \{1 \dots n\}}{[p_1, \dots, p_n] \prec [q_1, \dots, q_n \ \& \ r]}$
Spec-Map-Prefix	$\frac{p_1 \prec q_1 \quad \mathcal{L}(\{k_1 p_1, \dots\}) = \mathcal{L}(\{k_1 q_1, \dots\})}{\{k_1 p_1, \dots\} \prec \{k_1 q_1, \dots\}}$
Spec-Map-Tail	$\frac{p_1 \sim q_1 \quad \{k_2 \dots\} \prec \{k_2 \dots\}}{\{k_1 p_1, k_2 p_2, \dots\} \prec \{k_1 q_1, k_2 q_2, \dots\}}$
Spec-Keys	$\frac{\text{keys}(q) \subsetneq \text{keys}(p)}{p \prec q}$

for Level-2 atomic patterns via the class hierarchy DAG ($C_1 \sqsubset C_2 =$ transitive subclass relation). When two Level-2 patterns specialize on sibling classes (neither a subclass of the other), Spec-Type does not apply and $\prec_{\text{disp}}$ falls back to idx ; this is a known modularity hazard analogous to the Level-3 case. Spec-Prefix applies when first elements are strictly ordered ($p_1 \prec q_1$); Spec-Tail recurses on the remainder when heads are level-equal and share the same syntactic form ($p_1 \sim q_1$); the two rules are disjoint. Multi-parameter patterns apply the rules lexicographically position by position, after Spec-Level has partitioned patterns by max level.

The dispatch sort $\prec_{\text{disp}}$ executes at method-definition time (including during REPL re-evaluation). Let $\text{idx}(p)$ denote the definition index of the method clause containing p . FOL defines $\prec_{\text{disp}}$ via a three-level lexicographic sort key: ($\mathcal{L}(p)$ descending, $\prec$ -rank within level, $\text{idx}(p)$ descending). The partial order $\prec$ operates strictly within levels; idx breaks any remaining incomparabilities.

Exact-arity patterns of different lengths are incomparable; dispatch isolates the matching arity group first. For n arguments, this group contains all length- n fixed-arity patterns and length- $\leq n$ rest-accepting patterns. $\prec_{\text{disp}}$ is total over this group (Lemma 1). Spec-Fixed resolves the overlap by ordering the fixed-arity pattern

as more specific. Definition-order tie-breaking aligns with Lisp’s interactive workflow but introduces the modularity hazards discussed in Section 4.1.4.

LEMMA 4.1 (TOTALITY). $\prec_{\text{disp}}$ is a strict total order on arity-group patterns.

PROOF. $\prec$ is *irreflexive*: every rule requires a strict inequality (level, element position, key set, or subclass) that is false when $p_1 = p_2$. $\prec$ is *transitive*: within a level the rules encode a lexicographic comparison on position, arity, and key set. For sequential patterns: Spec-Tail applies only when both sequences have length ≥ 2 ; inductive step by Spec-Prefix when heads are strictly ordered, Spec-Tail when heads satisfy $p_1 \sim q_1$ with the inductive hypothesis on the remainder. If sequences are length 1 with $p_1 \sim q_1$, or if heads differ in form, they are incomparable under $\prec$ and resolved by *idx*. For map patterns: Spec-Map-Prefix and Spec-Map-Tail follow the same lexicographic argument over key-sorted position; Spec-Keys follows from transitivity of strict set containment ($\subsetneq$). Across levels, irreflexivity ensures at least one strict level inequality along any $p_1 \prec p_2 \prec p_3$ chain, so Spec-Level or within-level rules give $p_1 \prec p_3$. $\prec_{\text{disp}}$ is a *strict total order* by construction: it is defined by a sort algorithm and the explicit tiebreaker *idx*(*p*) ensures no two live clauses compare equal. Incomparability under $\prec$ arises when (i) patterns differ only in variable names, or (ii) sequential and map patterns share an arity. In both cases, the sort’s tiebreaker relies on *idx*, a monotonically increasing counter (re-evaluation increments it). Case (ii) is a modularity hazard: sequential and map clauses for the same generic function from different libraries are ordered solely by ASDF load sequence. $\square$

Predicate dispatch requires an $O(N)$ per-call scan. FOL sorts methods once at load time in $O(ND \log N)$. Subsumption systems achieve $O(1)$ cached dispatch via expensive load-time prover calls; FOL trades runtime caching for load-time syntactic tractability.

4.1.2 *Dispatch Examples*. Listing 1 shows a two-clause defn transpiled to a defun with a static cond cascade.

Listing 1: Transpilation: specificity-ordered multi-clause defn

```
;; FOL source: type specializer (Level 2) before wildcard (Level 0)
(defn classify
  ([x <integer>] :integer)
  ([x] :other))

;; Generated Common Lisp: ordering preserved as cond cascade
;; (class references bound once via load-time-value)
(defun classify (x)
  (cond
    ((typep x (load-time-value (find-class '<integer>))) :integer)
    (t :other)))
```

Comparison to defmulti. Clojure’s *defmulti* [17] requires manual resolution via *prefer-method*; FOL’s total order eliminates ambiguity for patterns with distinct types or structures.

4.1.3 *Usage Patterns*. Four patterns illustrate these features: structural diff (Listing 2); validated update guard (Listing 3); predicate dispatch on persistent state (Listing 4); and content routing (Listing 5).

Listing 2: Automatic Structural Diff (FOL)

```
(defmethod assoc :around [(obj <diffable>) key val]
  (if (= key :_changes)
    (call-next-method)
    ; key=:_changes: breaks cycle -- calls assoc
    ; directly, skipping the branch that emits
    ; (assoc ... :_changes ...) and re-invokes :around
    (bind [old (get obj key)
           new-obj (call-next-method)]
      (if (= old (get new-obj key))
        new-obj
        (assoc new-obj :_changes (inc (get obj :_changes 0)))))))
```

Listing 3: Validated Update Guard (FOL)

```
(defmethod assoc :around [(obj <guarded>) key val]
  (if (valid-update? obj key val)
    (call-next-method)
    obj)) ; discard invalid update; original snapshot unchanged
```

Listing 4: Predicate Dispatch on Persistent State (FOL)

```
;; Level-3 predicate (critical?) tests the reading value at dispatch time;
;; the body reads persistent slot :alert-history from the persistent sensor
;; object without copying. Both contributions are required: predicate
;; dispatch gates the clause; persistent immutability makes prior history
;; a first-class value.
(defmethod record-reading :around
  [(sensor <sensor>) (reading (critical?))]
  (bind [history (get sensor :alert-history [])
         result (call-next-method)]
    (assoc result :alert-history (conj history reading)))))
```

Listing 5: Value-Based Alerting (FOL)

```
(defmethod notify [{:keys [(temp (>= 100))]]]
  (print "ALERT: Critical Temperature!"))
```

Predicate forms evaluate their arguments dynamically at each dispatch call, responding to live variable updates (e.g., **limit**). However, mutating state inside the predicate closure corrupts dispatch correctness: the static sort relies on stable predicate structures.

FOL’s validated guard avoids the $O(N)$ copy required in CL (Listing 6). While Clojure can achieve per-type guards using multi-methods to emulate *:around*, doing so forces all clients of standard update! to migrate to the new multimethod; FOL’s *:around* integrates securely beneath the existing *assoc* generic function without restructuring the hierarchy.

Listing 6: Validated Update Guard in Common Lisp — requires explicit copy

```
(defmethod update-obj :around ((obj guarded) key val)
  ; No persistent objects: must copy before calling primary to preserve
  ; original for potential discard. Cost: O(N) in number of slots.
  (let ((saved (make-instance (class-of obj)
                              :slot-a (slot-value obj 'slot-a)
                              :slot-b (slot-value obj 'slot-b))))
    (if (valid-update-p obj key val)
        (call-next-method)
        saved)))
```

FOL centralizes the guard and discards updates in $O(1)$ via persistence.

4.1.4 *Design Limitations*. (1) The linear scan evaluates all non-matching predicates before a match; Level-3 predicates should be pure. (2) Load-order sensitivity arises for incomparable patterns (mixed Level-3 or structural types); collapse these into single clauses

or use explicit `:around` methods. (3) Open dispatch evaluates predicates sequentially; detecting unreachable clauses is future work. The MOP checks and warns of structurally incomparable Level-3 predicates at method definition.

4.2 Schema Evolution

Chain replay lazily composes alias maps, recovering renamed values in one pass. Migration confluence prohibits slot names appearing as both source and target, preventing non-terminating cycles or ambiguous diamond conflicts. Macro-expansion enforces these checks for `defclass` forms statically.

Listing 7: Lazy Schema Migration with Aliasing

```
;; Schema 1
(defclass <sensor> [] [[id] [reading]])
;; Schema 2: rename reading -> val; :reading keyword still routes to val
(defclass <sensor> [] [[id] [val :alias reading]])
```

At first post-redefinition *mutation*, chain replay writes `:reading`'s value into `:val`; reads of `:reading` before that mutation resolve via the alias map without modifying the instance.

4.2.1 Versioning Performance. Versioning theoretically costs $O(H \cdot A)$ (H =hops, A =aliases) constructing the map. The linear A dependence is bounded but unmeasured. Single-pass first-touch folding costs 3.6–8.2 μ s/instance across 1–3 hops (Tables 9–10). Subsequent reads/updates maintain standard 1.2–2.0 $\times$ overheads (Table 6).

5 Evaluation

5.1 Common Lisp and Clojure Comparison

FOL fills a gap between CLOS and Clojure (Table 5): FOL provides MOP-integrated persistence and full CLOS method combinations unavailable via `defrecord`.

Table 5: Built-in Feature Comparison

Feature	FOL	Clojure	CL
Static type safety	×	×	×
Persistent objects	✓ ^d	○ ^c	×
CLOS/MOP	✓	×	✓
Method combinations	✓	×	✓
Predicate dispatch	✓	single ^a	× ^b
Schema evolution	✓ ^e	×	Manual
Transducers	✓	✓	×
Macros	✓	✓	✓
Raw scalar performance	Low ^f	High	High

^a `defmulti` dispatches via a user-supplied function; `prefer-method/derive` allow manual overrides but there is no automatic multi-parameter specificity ordering.

^b Multi-parameter type dispatch natively; libraries like `trivia` add pattern matching.

^c Clojure has persistent *data structures*; `defrecord` is map-backed, not MOP-integrated.

^d Slot storage backed by immutable HAMTs; (`setf slot-value`) blocked; updates via `assoc`.

^e Renamed slots recovered automatically at first post-redefinition access; multi-hop chains composed in a single pass.

^f Scalar arithmetic and non-slot operations inherit full CL performance. “Low” refers specifically to persistent object slot access: reads add 1.2–2.0 $\times$ overhead and writes add 30–248 $\times$ overhead relative to mutable CLOS.

5.2 Micro-Benchmarks

We measured persistent objects vs. mutable CLOS across slot counts.

Construction (≈ 183 ns) is 5 $\times$ native; `%make-persistent` calls `allocate-instance` then (apply #'`initialize-instance` instance initargs), bypassing `make-instance` keyword-argument dispatch while preserving the full `initialize-instance` protocol, which stamps `%schema-version` and enforces the persistence invariant.

Reads add 1.2–2.0 $\times$; **Updates** are 30–248 $\times$ slower. Table 7 shows $T = 8$ is optimal for coverage of the fully-native path, as wider classes are rare.

Table 6: Actual slot update overhead (μ s/op) vs. native CLOS

Slots	Persistent	Mutable	Ratio	Notes
2	0.58 μ s	14.1 ns	41 $\times$	2 native slots
8	1.03 μ s	13.4 ns	77 $\times$	8 native slots
32	1.08 μ s	26.8 ns	40 $\times$	8N + 24V trie
48	2.23 μ s	24.0 ns	93 $\times$ ^a	8N + 40V trie
128	4.00 μ s	16.1 ns	248 $\times$	8N + 120V trie

^a Ratio drops vs. 32 slots because the mutable CLOS baseline is 46% slower at 48 slots (discontiguous memory layout), not because persistent overhead decreases.

Table 7: Modeled update cost (μ s/op) at candidate thresholds T

N	T=8	T=12	T=16	T=24	T=32	T=8 strategy
8	0.42	0.42	0.43	0.42	0.42	all-native
12	0.75	0.62	0.62	0.61	0.60	8N + 4V
16	0.94	0.91	0.83	0.78	0.77	8N + 8V
24	1.15	1.23	1.28	1.15	1.15	8N + 16V
32	1.39	1.47	1.51	1.58	1.52	8N + 24V
48	1.95	1.99	2.04	2.12	2.22	8N + 40V

Bold = minimum in row. “ $kN + jV$ ” = copy k native slots then batch-build j -element overflow vector. All-native = copy N slots directly. Simulated copy costs only; actual update-slots additionally incurs MOP iteration overhead (see Table 6).

5.2.1 Predicate Dispatch Micro-benchmark. We measured the per-invocation cost of a 20-method generic function, each clause specialized on a distinct Level-3 predicate, comparing FOL open and closed dispatch against standard CLOS type dispatch and a manual `cl:cond` cascade (Table 8, SBCL 2.6.0 x86-64).

Table 8: Predicate Dispatch Performance (N=20 clauses).

Dispatch Method	Time/op	Ratio (vs. CLOS)
Standard CLOS (Type)	37.4 ns	1.00 $\times$
Manual <code>cl:cond</code> Cascade	62.4 ns	1.66 $\times$
FOL <code>defn</code> (Closed Dispatch)	60.0 ns	1.60 $\times$
FOL <code>defgeneric</code> (Open Dispatch)	64.4 ns	1.72 $\times$

FOL’s predicate dispatch adds $\approx 1.6\times$ overhead, matching the `cl:cond` baseline; a 20-clause Level-3 setup approximates a worst case. Because open dispatch maps linearly without a type-indexed

cache, even all-Level-2 structures incur this $O(N)$ penalty. Overhead scales linearly with N ; $N = 50$ reaches $\approx 4\times$ CLOS.

5.2.2 Lazy Evolution Micro-Benchmark. We compared migration against a naive $O(A \cdot H)$ manual CL handler. FOL’s single-hop cost is $\approx 2\times$ CL’s. At greater depths (Table 10), SBCL’s version-history machinery dominates, confounding the empirical comparison of FOL’s automated $O(A)$ fold against the naive CL algorithm; however, FOL remains bounded at $\leq 8.2 \mu\text{s}$.

Table 9: Single-hop lazy migration: FOL vs. CL ($\mu\text{s/op}$, 10K instances)

Phase	FOL	CL	Notes
Migration (first touch)	6.40	3.30	one-time per dormant instance
of which:	5.63	3.28	SBCL migration + alias recovery
overhead vs. steady-state			
Steady-state update	0.77	0.02	post-migration, repeated use

Table 10: Multi-hop lazy migration first-touch cost ($\mu\text{s/op}$, 5K instances per cohort)

Hop depth	FOL	CL	Notes
1-hop ($v_2 \rightarrow v_3$)	8.19 ^b	4.10	FOL allocates new object; CL mutates in-place
2-hop ($v_1 \rightarrow v_3$)	3.57	5.62	FOL’s single-pass fold maintains $O(A)$ theoretical scaling.
3-hop ($v_0 \rightarrow v_3$)	4.48	5.13	SBCL migration machinery dominates overall time, confounding the algorithmic scaling difference.
Steady-state	0.69	0.01	post-migration; uniform across all cohorts

^b Higher than Table 9’s 6.40 μs : instances created at v_2 carry two prior class redefinitions in SBCL’s version history, incurring more migration overhead than instances created at v_1 or v_0 regardless of FOL hop depth. This explains the apparent anomaly that 1-hop costs more than 2-hop.

5.3 Naive-Translation Workloads

Persistent *slot* updates are 30–248 $\times$ slower than mutation (Table 6). Naive Quicksort and BFS (Listings 8–9) illustrate the path-copying penalty (Table 11).

Table 11: Naive-Translation Workload Results

Workload	Implementation	Time	Alloc	vs. CL
QuickSort $N = 100,000$	CL (simple-vector)	12 ms	0.8 MB	1.0 $\times$
	FOL Persistent (assoc)	1199 ms	2598 MB	$\sim 96\times$
	FOL Transient (assoc!)	494 ms	91 MB	$\sim 40\times$
BFS $N = 50,000$	CL (hash-table)	4 ms	2.3 MB	1.0 $\times$
	FOL Persistent (assoc)	148 ms	119 MB	39 $\times$
	FOL Transient (assoc!)	24 ms	13 MB	6.2 $\times$

Listing 8: Naive Persistent Partition

```
(defn partition [v low high]
  (bind [pivot (get v high)]
    (loop [j low i (dec low) curr-v v]
      (if (< j high)
        (if (<= (get curr-v j) pivot)
          (bind [next-i (inc i)]
            (v1 (assoc curr-v next-i (get curr-v j))
              v2 (assoc v1 j (get curr-v next-i))))
          (recur (inc j) next-i v2))
        (recur (inc j) i curr-v))
      (bind [next-i (inc i)]
        (v1 (assoc curr-v next-i (get curr-v high))
          v2 (assoc v1 high (get curr-v next-i)))
        [v2 next-i])))))
```

Listing 9: Naive Persistent BFS Step

```
(defn bfs-step [q dists graph]
  (if (empty? q)
    dists
    (bind [u (first q)]
      (edges (get graph u)
        d (get dists u)]
      (loop [es edges nq (rest q) nd dists]
        (if (empty? es)
          (bfs-step nq nd graph)
          (bind [v (first es)]
            (if (= (get nd v) -1)
              (recur (rest es)
                (conj nq v)
                (assoc nd v (inc d))))
              (recur (rest es) nq nd)))))))
```

Quicksort overhead stems from 3.4M path copies; transients cut allocation 97% but time only to 40 $\times$ due to dispatch and traversal overhead. BFS improves similarly (39 $\times \rightarrow$ 6.2 $\times$).

Idiomatic Reformulations. Algorithmic restructuring eliminates the translation penalty (Table 12). **Idiomatic merge sort** uses subvec and conj to minimize copying. **Lazy BFS** initializes nodes on discovery, bypassing N initial assoc calls.

Table 12: Idiomatic Algorithm Results: persistent and transient FOL vs. native CL

Workload	Implementation	Time	Alloc	vs. CL
Merge Sort $N = 100,000$	CL (simple-vector)	15 ms	1.6 MB	1.0 $\times$
	FOL persistent (conj)	457 ms	623 MB	30 $\times$
	FOL transient (HAMT array)	491 ms	81 MB	32 $\times$ ^a
BFS lazy $N = 50,000$	CL (hash-table)	4 ms	2.3 MB	1.0 $\times$
	FOL persistent (lazy dict)	103 ms	61 MB	29 $\times$
	FOL transient (hamt-assoc!)	51 ms	16 MB	14 $\times$

^a Transient is slower than persistent here: persistent conj touches only a root-to-leaf path of depth ≤ 5 per append, so transient setup cost is not recovered at $N = 100,000$.

Merge sort’s 1.7M trie-node touches (near $O(N \log^2 N)$) make transients counterproductive: the per-conj path is already shallow (depth ≤ 5), so the fixed cost of acquiring and committing a transient handle—thread-local ownership check plus dirty-node

finalization—is not recovered at $N = 100,000$. Lazy BFS trims initialization waste but its per-edge checks make it slower (14×) than naive pre-allocated transients (6.2×).

5.4 Abstract Syntax Tree Rewriting Engine

To evaluate both contributions together, we implemented an AST optimization engine: 25 node classes, a single multi-clause defmethod encoding 54 algebraic rewrite rules via predicate dispatch (including nested structural patterns such as the identity-element rule below), and a 25-clause walk driving recursive descent.

```
(defmethod ast-optimize
  [({:keys [(left ({:keys [(val (eq1 0))]) <op-lit>])
    (right r)]) <op-add>])
  r)
```

The built-in eq1 specialization (`(val (eq1 0))`) expands to `(lambda (x) (eq1 x 0))`, matching a slot against a specific value; it appears in the grammar’s predicate position. The CL baseline uses defstruct with typecase on a 9,999-node balanced tree.

Table 13 summarizes results. FOL is 43× slower per optimizer pass (14× more memory), driven by dispatch routing, slower field reads, and object construction. The build ratio (78×) exceeds both the optimizer ratio and the 5× per-object micro-benchmark because: (1) the CL baseline uses defstruct, which is faster than the make-instance baseline in the micro-benchmark; and (2) constructing 9,999 nodes runs the full initialize-instance MOP protocol per node—stamping %schema-version, populating the base object, and initializing the overflow trie—with no amortization across the tree, accumulating GC pressure. Transient builders (Section 3) are expected to reduce the allocation component.

Table 13: AST Optimizer: FOL vs. CL ($N = 100$ passes, 9,999-node balanced tree).

Phase	Implementation	Time	Alloc	vs. CL
Build	CL	0.398 ms	0.16 MB	1.0×
	FOL	31.1 ms	12.7 MB	78×
Optimizer (per pass)	CL	0.240 ms/pass	0.125 MB/pass	1.0×
	FOL	10.36 ms/pass	1.749 MB/pass	43×

5.5 Logic Simulation (LSim)

LSim is an event-driven digital logic simulator that exercises persistent data structures at scale. Gate-connectivity is held constant; implementations differ in both event-queue and gate-state storage: the CL baseline uses a mutable destructive leftist heap [9] with mutable hash-table gate state; FOL uses a persistent leftist heap [24] with HAMT gate state. The 2×2 ablation (Table 15) isolates the individual contributions of each component. Table 14 covers six configurations spanning five orders of magnitude; the largest circuits used a 56 GB heap (–dynamic-space-size 57344).

At scale, CL mutable hash-table allocation scatters entries across the heap, degrading GC locality; persistent HAMTs avoid this by keeping internal nodes short-lived and depth-bounded, yielding FOL’s 3.8× advantage at the largest configuration (averaged over three iterations).

Table 14: LSim PQ Scaling Results (mean wall time).

Config	Gate Evals	CL	FOL	Ratio (FOL/CL)
8bit-100	876	78 ms	109 ms	1.4×
32bit-300	10 224	94 ms	141 ms	1.5×
8x32-9000	281 248	359 ms	1.87 s	5.2×
32x32-3000	3 850 304	7.57 s	34.15 s	4.5×
8x32x32-9000	89 708 032	906 s	983 s	1.1×
32x32x32-30000	1,183,510,528	50,007 s	13,201 s	0.26× ^a

^a Small-circuit times include shared ≈74 ms initialization overhead; ratio < 1 means FOL is faster. The largest config was ablated (Table 15); the fragmentation explanation is supported by the 2×2 breakdown where persistent maps avoid hash-table degradation.

Table 15: 2×2 Ablation: Mutable vs. Persistent Heap × State Maps (mean wall time; ratios relative to CL baseline). Hybrid-A = persistent maps + mutable heap; Hybrid-B = mutable maps + persistent heap; FOL = both persistent.

Circuit	CL	Hybrid-A	Hybrid-B	FOL
32bit-300	94 ms	141 ms (1.5×	125 ms (1.3×	141 ms (1.5×
8x32-9000	359 ms	1.87 s (5.2×	2.25 s (6.3×	1.87 s (5.2×
32x32-3000	7.57 s	34.1 s (4.5×	42.7 s (5.6×	34.1 s (4.5×
8x32x32-9000	906 s	983 s (1.1×	1646 s (1.8×	983 s (1.1×
32x32x32-30000	50.0k s	13.2k s (0.26×	67.5k s (1.35×	13.2k s (0.26×

A 2×2 ablation (Table 15) breaks down these costs.

FOL matches Hybrid-A across ablated runs: HAMT allocation dominates the heap, rendering persistent heap additions negligible. Hybrid-A/FOL overhead peaks mid-scale (5–6×) then recovers. Hybrid-B falls from 6.3× to 1.35×; as scale grows, the CL baseline itself severely degrades from hash-table fragmentation, making Hybrid-B’s fixed $O(\log n)$ heap tax look comparatively better. At 32x32x32-30000, FOL is 3.8× faster. This gain stems purely from state maps: SBCL’s hash-tables scatter entries, bloating GC scans. HAMTs avoid this: gate-state tables hold ≤ 32 entries, bounding trie depth to ≤ 2 nodes that stay within nursery allocations.

5.6 CLOS Adoption Cost: Expression Interpreter

We port an idiomatic CLOS expression interpreter (seven node types, three generic functions: eval-expr, pretty, free-vars) to FOL with minimal structural changes. The CL baseline uses mutable environments; FOL uses (assoc env bvar v); method bodies are otherwise identical.

A corpus of 50 depth-5 expression trees is evaluated 1,000 times (150,000 root calls, yielding ≈9.4M recursive generic function dispatches).

Results. The 7× eval ratio (vs. 43× for the AST) reflects: a make-instance baseline, fewer overall clauses per scan (7 vs. 25), and crucially, Level-2 type specializers checking classes statically without the expensive function-call evaluation that the AST’s Level-3 structural predicates rigidly impose across bypassed targets. The port requires no architectural logic shifts; differences lie purely in immutable mappings.

Table 16: Expression Interpreter Benchmark (SBCL 2.6.0, AMD Ryzen 9 5900X).

Phase	CL	FOL	Time ratio	Mem ratio
Build (50 trees, depth 5)	9.6 ms / 3.8 MB	19.9 ms / 8.9 MB	2.1×	2.3×
Eval (1000 iter × 50 exprs)	0.27 s / 224.5 MB	1.88 s / 1300 MB	7.0×	5.8×

6 Threats to Validity

Single implementation: All measurements use SBCL 2.6.0; results may differ on other CL implementations. **Experimenter bias:** FOL and all baselines share the same author; more aggressive CL tuning might narrow results. **Benchmark coverage:** Smaller LSim configs were run repeatedly; the largest circuit was run three times per implementation due to its scale (≈ 14 hours per run). **Benchmark selection bias:** Macro-benchmarks may over-represent FOL’s advantages; the interpreter and naive-translation benchmarks offer a more conservative view. **Qualitative comparison:** Section 4 compares code-level conciseness but does not measure programmer productivity. **Interoperability hazard:** CL authors extending FOL classes may encounter runtime mutation errors; these are not signaled at method-definition time. **Non-portability:** persistent-class uses standard AMOP hooks (allocate-instance, initialize-instance), but the native slot fast-path reads from SBCL’s internal object-header layout directly; portability to other runtimes (e.g., CCL, ECL, ABCL) is untested. **Dispatch scaling:** The predicate dispatch micro-benchmark uses a single $N = 20$ configuration; overhead is projected to scale linearly with N , but this scaling is not empirically verified across multiple clause counts. **Re-evaluation ordering instability:** Definition index is a monotonically increasing counter; REPL re-evaluation of any clause assigns it the highest current index, silently reordering all incomparable clause pairs that involve the re-evaluated clause. Method ordering can therefore differ between two sessions that load the same files in the same order if interactive re-evaluation occurred in either session.

7 Related Work

Persistent data structures. FOL’s trie follows Bagwell’s HAMT [3] and Clojure [17], using path-copying persistence [11]; Okasaki’s heaps [24] underpin our LSim heap. FOL extends libraries like Sycamore [10] and FSet [5] to persistent *objects* with predicate dispatch.

Schema evolution. Gemstone [27] and ENCORE [29] pioneered lazy migration, but require explicit coercion code for renames [2]. FOL provides declarative migration via `:alias`; multi-hop chains compose in a single pass via the CLOS MOP.

OOP/FP bridges and pattern dispatch in other languages. Scala’s case classes [23] and match [12] provide immutable ADTs with structural dispatch, but updates require explicit copy and the class system is not MOP-integrated. Racket [14] separates match from its class system [15]; Julia [4] and MultiJava [8] provide multi-parameter type dispatch without predicate specializers.

Expression Problem solutions. Object algebras [25] and tagless-final style [6] solve both dimensions in typed languages with static guarantees. FOL’s approach is a multimethod-based solution [32]

that trades static safety for CLOS expressibility and open-world extension.

Predicate dispatch. JPred [13, 22] requires a theorem prover for ordering. Later Cecil work [7] achieves efficient predicate dispatch via aggressive whole-program decision-tree compilation, contrasting with FOL’s open-world syntactic ordering without a prover. Our composite level recovers ranking for type constructs statically. GHC’s overlapping instances [16] order heads via pragmas but lack a predicate stratum or evolution. Filtered dispatch [26] introduced guard predicates as first-class criteria, motivating FOL’s Level-3. Dylan’s singletons [1] and Slate’s dispatch [28] are restricted forms. Clojure’s `defmulti` [17] dispatches via a user-supplied function with `prefer-method` for ambiguity; *protocols* [18] dispatch on the first argument’s class only. FOL’s `defmethod` occupies the same niche as `defmulti` but adds predicate specializers and CLOS method combinations, trading an $O(N)$ scan for expressivity. Self’s [30] “slots as map entries” model influences FOL’s object representation.

8 Conclusion

We presented two contributions to the design of Lisp object systems, supporting open extension and addressing the new-operations dimension of the Expression Problem within a dynamically typed setting. First, a **persistent metaclass system** built on the CLOS MOP: hybrid native/trie storage keeps $\approx 80\%$ of classes on the 1.2–2.0× read-overhead native path; lazy alias-based migration delivers transparent instance evolution; and `%make-persistent` reduces per-object construction cost to 5× over mutable `make-instance` (micro-benchmark; build-phase ratios are lower when traversal dominates). Second, a **specificity-ordered predicate dispatch** whose load-time-decidable $\prec_{\text{disp}}$ orders structurally differentiated patterns, confining load-order sensitivity to incomparable Level-3 predicates, sibling-class Level-2 types, and structural ties.

Slot updates cost 30–248× in isolation; FOL is thus a scale-out solution for complex, versioned architectures rather than a latency-optimized replacement for mutable slots. Its advantage emerges at scale: at 1.18 billion gate evaluations, FOL is 3.8× faster than CL as the mutable heap degrades. The AST optimizer pass is 43× slower; speculative rollback is $O(1)$ by construction—discarding an assoc result returns the original snapshot without copying—but is not separately benchmarked.

Open problems include: caching techniques to mitigate the $O(N)$ predicate scan without a theorem prover; static analysis to detect unreachable patterns; and coordinated transitions across agent networks.

FOL’s implementation is available at <https://github.com/frankadrian314159/fol>.

References

- [1] Apple Computer, Inc. 1992. *Dylan - An Object-Oriented Dynamic Programming*

- Language. Apple Computer, Inc., Cupertino, CA, USA.
- [2] Malcolm Atkinson and Ron Morrison. 1995. Orthogonally Persistent Object Systems. *The VLDB Journal* 4, 3 (1995), 319–401.
- [3] Phil Bagwell. 2001. *Ideal Hash Trees*. Technical Report 1. EPFL, Lausanne, Switzerland.
- [4] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Rev.* 59, 1 (2017), 65–98.
- [5] Scott L Burke. 2007. FSet: A functional set-theoretic collections library. <https://common-lisp.net/project/fset/>
- [6] Jacques Carette, Oleg Kiselyov, and Chung-chieh Shan. 2009. Finally Tagless, Partially Evaluated: Tagless Staged Interpreters for Simpler Typed Languages. *Journal of Functional Programming* 19, 5 (2009), 509–543.
- [7] Craig Chambers and Weimin Chen. 1999. Efficient Multiple and Predicate Dispatching. In *Proceedings of the 14th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 238–255.
- [8] Curtis Clifton, Gary T. Leavens, Craig Chambers, and Todd Millstein. 2000. MultiJava: Modular Open Classes and Symmetric Multiple Dispatch for Java. In *Proceedings of the 15th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*. ACM, 130–145.
- [9] Clark Allen Crane. 1972. *Linear Lists and Priority Queues as Balanced Binary Trees*. Technical Report STAN-CS-72-259. Stanford University.
- [10] Neil T. Dantam. 2013. Sycamore: Purely Functional Data Structures for Common Lisp. <https://github.com/ndantam/sycamore>
- [11] James R Driscoll, Neil Sarnak, Daniel D Sleator, and Robert E Tarjan. 1989. Making Data Structures Persistent. *J. Comput. System Sci.* 38, 1 (1989), 86–124.
- [12] Burak Emir, Martin Odersky, and John Williams. 2007. Matching Objects with Patterns. In *ECOOP 2007—Object-Oriented Programming*. Springer, Berlin, Heidelberg, 273–298.
- [13] Michael D Ernst, Craig Kaplan, and Craig Chambers. 1998. Predicate Dispatching: A Unified Theory of Dispatch. In *ECOOP’98—Object-Oriented Programming*. Springer, Berlin, Heidelberg, 186–211.
- [14] Matthew Flatt et al. 2015. The Racket Manifesto. In *20 Years of the Past and 20 Years of the Future (Snapshots)*, Dagstuhl Seminar 15451. <https://doi.org/10.4230/DagRep.5.11.112>
- [15] Matthew Flatt, Robert Bruce Findler, and Matthias Felleisen. 2006. Scheme with Classes, Mixins, and Traits. In *Asian Symposium on Programming Languages and Systems*. Springer, 270–289.
- [16] GHC Team. 2004. *GHC User’s Guide: Overlapping Instances*. Glasgow Haskell Compiler. https://downloads.haskell.org/ghc/latest/docs/users_guide/.
- [17] Rich Hickey. 2008. The Clojure programming language. In *Proceedings of the 2008 symposium on Dynamic languages*. ACM, New York, NY, USA, 1–10.
- [18] Rich Hickey. 2010. Clojure Protocols. Clojure Reference Documentation. <https://clojure.org/reference/protocols>
- [19] Rich Hickey. 2010. Clojure’s Solutions to the Expression Problem. Conference talk at Strange Loop. Available at [the strangeloop.com](https://www.thestrangeloop.com/2010/clojures-solutions-to-the-expression-problem.html).
- [20] Gregor Kiczales, Jim Des Rivieres, and Daniel Gureasko Bobrow. 1991. *The Art of the Metaobject Protocol*. MIT press.
- [21] Michele Lanza and Radu Marinescu. 2006. *Object-Oriented Metrics in Practice*. Springer.
- [22] Todd Millstein. 2004. Practical Predicate Dispatch. In *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*. ACM, 345–364.
- [23] Martin Odersky, Lex Spoon, and Bill Venners. 2016. *Programming in Scala, 3rd Edition*. Artima Press.
- [24] Chris Okasaki. 1998. *Purely Functional Data Structures*. Cambridge University Press.
- [25] Bruno C. d. S. Oliveira and William R. Cook. 2012. Extensibility for the Masses: Practical Extensibility with Object Algebras. In *Proceedings of the 26th European Conference on Object-Oriented Programming (ECOOP)*. Springer, 2–27.
- [26] Doug Orleans. 2002. Filtered Dispatch. In *Proceedings of the 17th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. ACM, New York, NY, USA, 20–26.
- [27] D. J. Penney and J. Stein. 1987. Class Modification in the GemStone Object-Oriented DBMS. In *Proceedings of the 2nd ACM Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*. ACM, 111–117.
- [28] Lee Salzman and Jonathan Aldrich. 2005. Prototypes with Multiple Dispatch: An Expressive and Dynamic Object Model. In *ECOOP 2005—Object-Oriented Programming*. Springer, 312–336.
- [29] Andrea H. Skarra and Stanley B. Zdonik. 1986. The Management of Changing Types in an Object-Oriented Database. In *Proceedings of the 1st ACM Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*. ACM, 483–495.
- [30] David Ungar and Randall B. Smith. 1987. Self: The Power of Simplicity. *ACM SIGPLAN Notices* 22, 12 (1987), 227–242.
- [31] Philip Wadler. 1990. Linear Types Can Change the World!. In *Programming Concepts and Methods*. North-Holland, Amsterdam, Netherlands, 347–359.
- [32] Matthias Zenger and Martin Odersky. 2005. Independently Extensible Solutions to the Expression Problem. In *Proceedings of the 12th International Workshop on Foundations of Object-Oriented Languages (FOOL)*.